1 · BOUNDED BUFFER
Problem
Diagram
Solution
FreeRTOS Code
2 · READERS–WRITERS
Problem
R1 / R2 Variants
Diagram
FreeRTOS Code
3 · DINING PHILOSOPHERS
Problem
Deadlock
Solutions
FreeRTOS Code
4 · SLEEPING BARBER
Problem
Diagram
FreeRTOS Code
5 · MONITORS
Concept
Condition Variables
FreeRTOS Analogy
Monitor: Bounded Buffer

01 Bounded Buffer (Producer–Consumer)

A fixed-size circular buffer is shared between one or more **producers** (who insert items) and one or more **consumers** (who remove items). Three constraints must hold simultaneously:

- A producer must wait when the buffer is **full**.
- A consumer must wait when the buffer is **empty**.
- Only one actor may modify the buffer at a time (**mutual exclusion**).

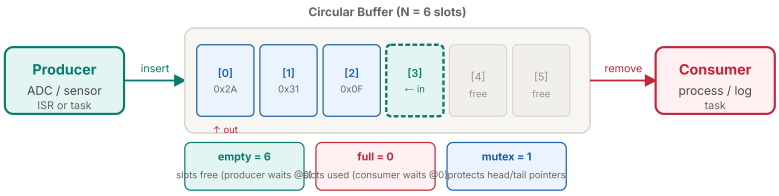


Figure 1 — Bounded buffer: 6-slot circular ring with producer/consumer and three semaphores

Solution — Three Semaphores

Semaphore	Initial value	Producer calls	Consumer calls	Purpose
`empty`	N (buffer size)	Take (before insert)	Give (after remove)	Counts free slots
`full`	0	Give (after insert)	Take (before remove)	Counts filled slots
`mutex`	1	Take/Give around buffer access	Take/Give around buffer access	Mutual exclusion on head/tail

FreeRTOS Code

```
#define N  8
static uint8_t buf[N];
static int head = 0, tail = 0;
```

```
SemaphoreHandle_t empty, full, mutex;

void init(void) {
    empty = xSemaphoreCreateCounting(N, N); /* N free slot
    full  = xSemaphoreCreateCounting(N, 0); /* 0 filled sl
    mutex = xSemaphoreCreateMutex();
}

/* ── Producer ──────────────────────────────────────── */
void Producer(void *p) {
    for (;;) {
        uint8_t item = produce_item();

        xSemaphoreTake(empty, portMAX_DELAY);  /* wait for
        xSemaphoreTake(mutex, portMAX_DELAY);  /* lock buf
        buf[head] = item;
        head = (head + 1) % N;
        xSemaphoreGive(mutex);                 /* unlock
        xSemaphoreGive(full);                  /* signal i
    }
}

/* ── Consumer ──────────────────────────────────────── */
void Consumer(void *p) {
    for (;;) {
        xSemaphoreTake(full,  portMAX_DELAY);  /* wait for
        xSemaphoreTake(mutex, portMAX_DELAY);  /* lock buf
        uint8_t item = buf[tail];
        tail = (tail + 1) % N;
        xSemaphoreGive(mutex);                 /* unlock
        xSemaphoreGive(empty);                 /* signal a

        consume_item(item);
    }
}
```

> ⚠ **ORDER MATTERS**
>
> Take `empty` / `full` *before* taking `mutex`. If you take `mutex` first and then block on `empty`, you hold the lock while sleeping — the other party can never give back `empty` because it's also waiting for `mutex`. Deadlock.

02 Readers–Writers Problem

Multiple tasks share a data structure (a database record, a sensor cache). **Readers** only inspect it — multiple readers may run concurrently. **Writers** modify it — a writer needs

exclusive access; no reader or other writer may run simultaneously.

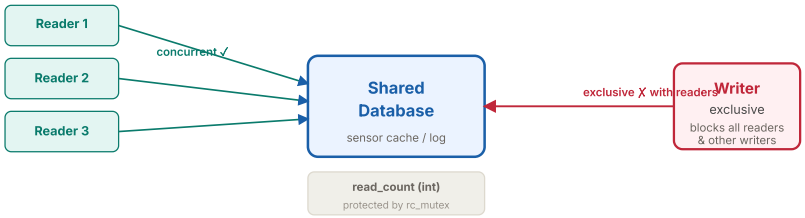


Figure 2 — Multiple readers share concurrently; a writer gets exclusive access

Two Classic Variants

FIRST READERS PREFERENCE (R1)

Once even one reader is active, new readers are allowed in immediately. Writers wait until *all* readers leave. Risk: **writer starvation** — a stream of readers can indefinitely block the writer.

FIRST WRITERS PREFERENCE (R2)

Once a writer is waiting, no new readers start. Waiting readers queue behind the writer. Risk: **reader starvation** — a burst of writers starves readers. In practice, R1 is most commonly implemented.

FreeRTOS Code (R1 — Readers Preference)

```
static int read_count = 0;
SemaphoreHandle_t rc_mutex;   /* protects read_count */
SemaphoreHandle_t rw_mutex;   /* exclusive write / first-r

void rw_init(void) {
    rc_mutex = xSemaphoreCreateMutex();
    rw_mutex = xSemaphoreCreateBinary();
    xSemaphoreGive(rw_mutex);
}

/* ── Reader ──────────────────────────────────────── */
void Reader(void *p) {
    for (;;) {
        /* Entry: increment reader count; first reader loc
        xSemaphoreTake(rc_mutex, portMAX_DELAY);
        read_count++;
        if (read_count == 1)
            xSemaphoreTake(rw_mutex, portMAX_DELAY); /* fi
        xSemaphoreGive(rc_mutex);

        /* ── critical section: read shared data ── */
        read_data();
```

```
        /* Exit: decrement reader count; last reader unloc
        xSemaphoreTake(rc_mutex, portMAX_DELAY);
        read_count--;
        if (read_count == 0)
            xSemaphoreGive(rw_mutex); /* last reader unloc
        xSemaphoreGive(rc_mutex);

        vTaskDelay(pdMS_TO_TICKS(10));
    }
}

/* — Writer ———————————————————————————————— */
void Writer(void *p) {
    for (;;) {
        xSemaphoreTake(rw_mutex, portMAX_DELAY); /* full e
        write_data();
        xSemaphoreGive(rw_mutex);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

KEY INSIGHT

`rw_mutex` is taken by the *first reader* and released by the *last reader* — not by the same entity. This is why a binary semaphore works here but a mutex (owner-tracked) cannot be used for `rw_mutex`.

03 Dining Philosophers

Five philosophers sit at a round table. Between each adjacent pair lies one fork — five forks total. A philosopher alternates between thinking and eating. To eat, she must pick up *both* the fork on her left and the fork on her right. After eating, she puts both forks down.

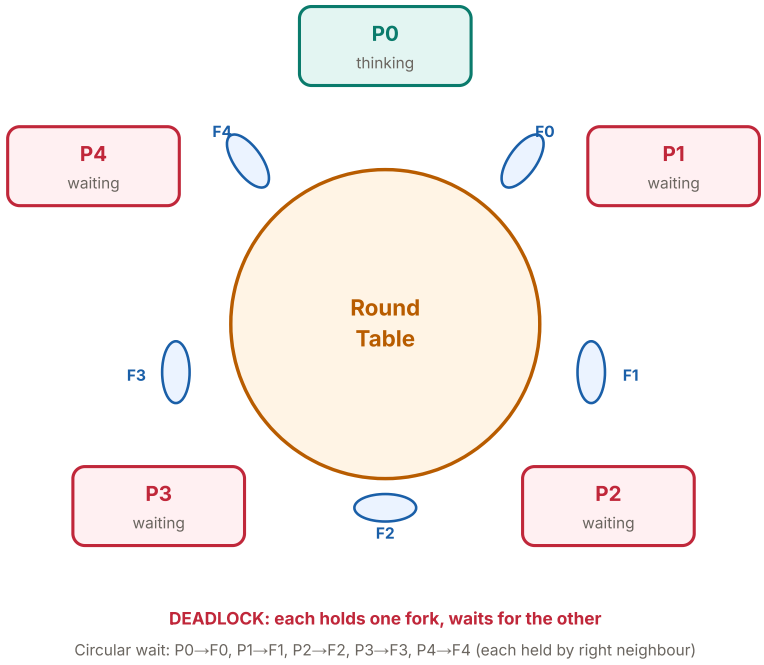


Figure 3 — Five philosophers; naive "pick left first" causes circular wait deadlock

Solutions

✅ ASYMMETRIC / RESOURCE ORDERING

One philosopher (P4) picks up the *right* fork first instead of the left. This breaks the circular wait. At least one philosopher can always eat.

ARBITRATOR (WAITER) PATTERN

A mutex-protected waiter grants permission to pick up both forks atomically. Only four philosophers may attempt to eat simultaneously (counting semaphore init=4 for N=5 table).

FreeRTOS Code (Asymmetric)

```
#define N 5
SemaphoreHandle_t fork[N];

void phil_init(void) {
    for (int i = 0; i < N; i++)
        fork[i] = xSemaphoreCreateMutex();
}

void Philosopher(void *arg) {
    int id = (int)(intptr_t)arg;
    int left  = id;
    int right = (id + 1) % N;
```

```
    for (;;) {
        /* Think */
        vTaskDelay(pdMS_TO_TICKS(200 + (id * 37)));

        /* Pick up forks — P4 reverses order to break circ
        if (id == (N - 1)) {
            xSemaphoreTake(fork[right], portMAX_DELAY);
            xSemaphoreTake(fork[left],  portMAX_DELAY);
        } else {
            xSemaphoreTake(fork[left],  portMAX_DELAY);
            xSemaphoreTake(fork[right], portMAX_DELAY);
        }

        /* Eat */
        vTaskDelay(pdMS_TO_TICKS(100));

        /* Put forks down */
        xSemaphoreGive(fork[left]);
        xSemaphoreGive(fork[right]);
    }
}
```

04 Sleeping Barber

A barbershop has one barber, one barber chair, and a waiting room with **N** seats. If no customers are present, the barber sleeps. A customer who arrives:

- Wakes the barber if he is asleep.
- Sits in a waiting seat if the barber is busy and seats are free.
- **Leaves** if all waiting seats are taken.

This models any server that should sleep when idle and handle bounded-queue overflow gracefully — a thread pool, a UART Rx handler, a network packet processor.

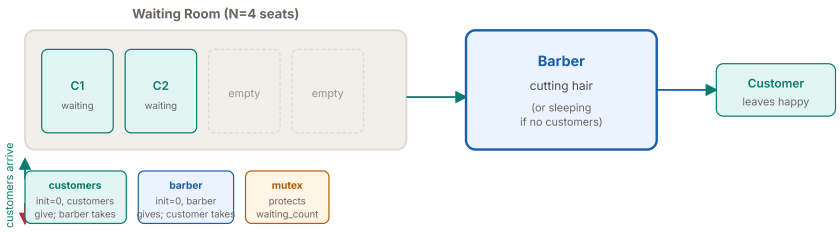


Figure 4 — Sleeping Barber: waiting room with N=4 seats; barber sleeps when idle

FreeRTOS Code

```
#define N_SEATS  4
static int waiting = 0;
```

```
SemaphoreHandle_t customers; /* barber sleeps on this (ini
SemaphoreHandle_t barber;    /* customer waits for barber
SemaphoreHandle_t sb_mutex;  /* protects 'waiting' count *

void sb_init(void) {
    customers = xSemaphoreCreateCounting(N_SEATS, 0);
    barber    = xSemaphoreCreateBinary();
    sb_mutex  = xSemaphoreCreateMutex();
}

/* ── Barber task (single) ──────────────────────────── *
void BarberTask(void *p) {
    for (;;) {
        xSemaphoreTake(customers, portMAX_DELAY); /* sleep
        xSemaphoreTake(sb_mutex,  portMAX_DELAY);
        waiting--;
        xSemaphoreGive(barber);    /* invite one customer
        xSemaphoreGive(sb_mutex);
        cut_hair();
    }
}

/* ── Customer task (many instances / ISR-spawned) ──────
void CustomerTask(void *p) {
    xSemaphoreTake(sb_mutex, portMAX_DELAY);
    if (waiting < N_SEATS) {
        waiting++;
        xSemaphoreGive(customers); /* wake / queue for bar
        xSemaphoreGive(sb_mutex);
        xSemaphoreTake(barber, portMAX_DELAY); /* wait for
        get_haircut();
    } else {
        xSemaphoreGive(sb_mutex);
        /* waiting room full — leave immediately */
    }
    vTaskDelete(NULL);
}
```

EMBEDDED ANALOGY

Replace "barber" with a **UART DMA ISR handler task**. The barber is the processing task (blocked when idle), customers are incoming DMA-complete interrupts, the waiting room is a bounded message queue. Customers that arrive when the queue is full are simply dropped — acceptable packet loss behaviour.

05 Monitors

A **monitor** is a higher-level synchronisation construct that bundles a shared data structure, its mutex, and a set of *condition variables* into a single module. The language or runtime guarantees mutual exclusion automatically — the programmer cannot forget to lock.

Introduced by C.A.R. Hoare (1974) and popularised by Java's `synchronized` blocks, C++17's `std::condition_variable`, and POSIX `pthread_mutex` + `pthread_cond`.

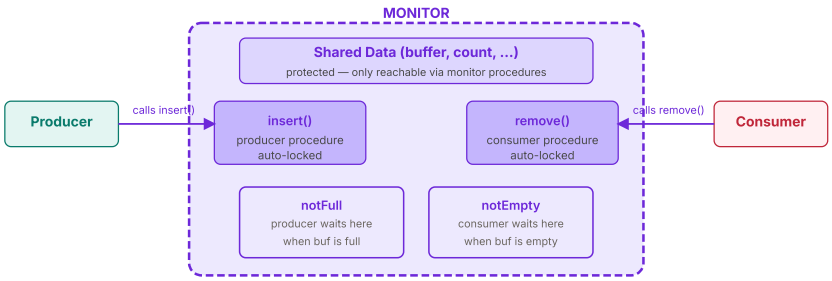


Figure 5 — Monitor structure: shared data + procedures + condition variables, all in one unit

Condition Variables

A condition variable (CV) has two operations:

Operation	Effect	POSIX
wait(cv, mutex)	Atomically release mutex + block on cv. Re-acquires mutex when woken.	`pthread_cond_wait`
signal(cv)	Wake one thread waiting on cv (no-op if none waiting).	`pthread_cond_signal`
broadcast(cv)	Wake all threads waiting on cv.	`pthread_cond_broadcast`

⚠ ALWAYS CHECK CONDITION IN A LOOP

Spurious wakeups can occur (the OS may wake a thread without a signal). Always test the condition after being woken:

```
while (buffer_full())          /* NOT if() */
    pthread_cond_wait(¬Full, &mutex);
```

Monitor in FreeRTOS

FreeRTOS does not have a monitor primitive, but you build one with a **mutex + Task Notifications or EventBits**. The mutex provides the automatic lock; task notification or a binary semaphore acts as the condition variable signal.

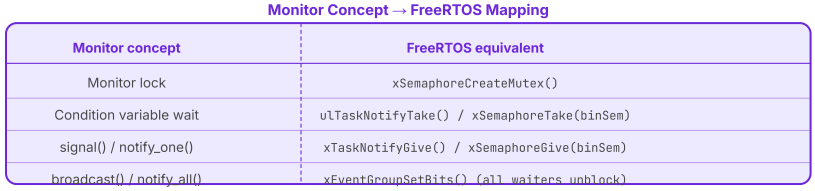


Figure 6 — FreeRTOS equivalents for the monitor's built-in primitives

Monitor-Style Bounded Buffer in C (POSIX)

```
/* monitor_bb.c — bounded buffer as a monitor (POSIX pthre
#define N 8
static int buf[N], head = 0, tail = 0, count = 0;
static pthread_mutex_t mon  = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t  notFull  = PTHREAD_COND_INITIALIZER
static pthread_cond_t  notEmpty = PTHREAD_COND_INITIALIZER

/* ── insert() — monitor entry procedure ── */
void monitor_insert(int item) {
    pthread_mutex_lock(&mon);           /* enter monitor —

    while (count == N)                  /* wait on conditi
        pthread_cond_wait(¬Full, &mon);

    buf[head] = item;
    head = (head + 1) % N;
    count++;

    pthread_cond_signal(¬Empty);     /* signal consumer */
    pthread_mutex_unlock(&mon);         /* exit monitor —
}

/* ── remove() — monitor entry procedure ── */
int monitor_remove(void) {
    pthread_mutex_lock(&mon);

    while (count == 0)
```

```
        pthread_cond_wait(¬Empty, &mon);

    int item = buf[tail];
    tail = (tail + 1) % N;
    count--;

    pthread_cond_signal(¬Full);      /* signal producer */
    pthread_mutex_unlock(&mon);
    return item;
}
```

WHY MONITORS ARE BETTER THAN RAW SEMAPHORES

- The lock is **implicit** — a programmer cannot accidentally read shared data outside the monitor.
- `wait(cv)` **atomically** releases the lock and suspends — no window for a missed signal between "unlock" and "block".
- Condition variables express the *reason* for waiting (`notFull` , `notEmpty`) rather than embedding it in semaphore count arithmetic.
- Code is easier to audit: every shared-data access is inside a named procedure.

Side-by-Side Summary

Problem	Semaphores used	Key invariant	Classic pitfall
Bounded Buffer	`empty` , `full` , `mutex`	Take empty/full *before* mutex	Deadlock if mutex taken first
Readers–Writers	`rc_mutex` , `rw_mutex`	First/last reader gates `rw_mutex`	Writer starvation (R1) or reader starvation (R2)
Dining Philosophers	One mutex per fork	Asymmetric pick-up for one philosopher	Circular wait if all pick left fork first
Sleeping Barber	`customers` , `barber` , `mutex`	Overflow path (leave if full) inside mutex	Race on `waiting` count outside mutex
Monitor	1 mutex + condition variables	Always `while()` not `if()` on CV wait	Spurious wakeup or missed signal